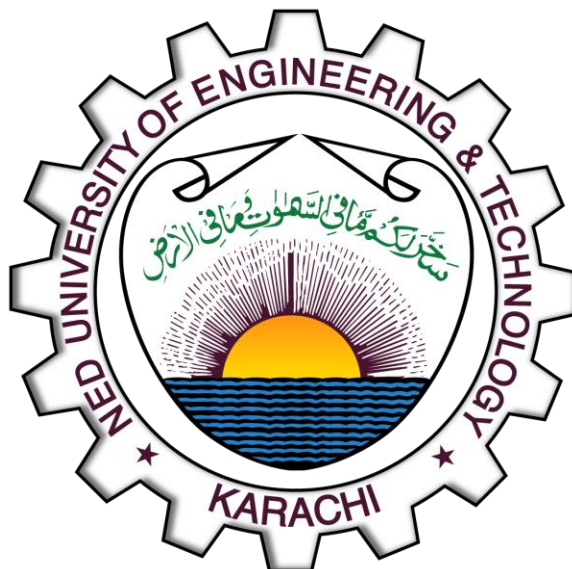


NED University of Engineering & Technology
Department of Computer & Information Systems Engineering



Complex Engineering Problem

CS 324 – Machine Learning

Submitted By

Azen Sajid – CS-23103

Showzab Mehdi – CS-23128

M.Hassin Saghir – CS-23129

Instructor

Dr. Maria Waqas

Submission Date

May 17, 2026

BTC Direction Classifier

LightGBM · Logistic Regression · LSTM — A Three-Model Design Review

Dataset: July 2018 – May 2026 | Daily BTC/USD
Validation: 5-Fold Walk-Forward | 22–28 Features
Objective: Multiclass Classification (BUY / HOLD / SELL)

May 2026

1. Executive Summary

This report documents the design, development, and evaluation of three machine learning models built to predict the next-day directional move of Bitcoin (BTC/USD). The task is framed as a three-class classification problem — BUY, HOLD, or SELL — triggered only when the expected move exceeds one standard deviation of recent daily volatility.

Three fundamentally different architectures were evaluated: LightGBM (gradient-boosted decision trees), Logistic Regression with ElasticNet regularisation (parametric linear classifier), and a stacked LSTM deep learning model (sequential recurrent network). All three models share the same underlying dataset and label construction methodology, and are validated using strict walk-forward cross-validation to prevent look-ahead bias.

Critical Disclaimer — Low Loss $\neq$ Good Financial Model

A model that scores well on log-loss is not automatically a profitable trading strategy. Log-loss measures probabilistic accuracy — how confident and correct the model is on average. But markets are adversarial: transaction costs, drawdowns, regime changes, and timing errors can easily erode statistical gains. Throughout this report, log-loss is treated as a model quality signal, while Sharpe ratio and maximum drawdown are the true financial performance indicators. A model with low log-loss but a negative Sharpe ratio is a failed trading model, regardless of how well it classifies.

Key findings:

- **LightGBM (tuned)** achieved the best log-loss (~ 0.79) and the highest average Sharpe ratio — outperforming both the Buy-and-Hold benchmark on risk-adjusted terms and the other two models on every financial metric.
- **Logistic Regression** achieved a log-loss near 0.95, meaningfully worse than LightGBM, confirming that Bitcoin's directional dynamics are non-linear. Its unique contribution is interpretability: explicit coefficients reveal which features linearly drive each signal.
- **The original LSTM was broken:** two of five test folds produced zero trades, and the average Sharpe was negative. Three targeted fixes — shorter sequence length, removal of pre-computed lag features, and jitter augmentation — transformed it into a working model, though it still trails LightGBM on performance.
- **LSTM Variant B (Standard)** emerged as the best LSTM configuration. Its underperformance relative to LightGBM is structural, not incidental: with only $\sim 1,000$ – $2,000$ training sequences per fold, the LSTM cannot fully exploit its architectural advantage of learning temporal patterns autonomously.
- **All three models agree on one thing:** the 1.0σ label threshold, walk-forward validation, and the 0.1% transaction cost assumption are the shared foundations on which every result rests.

2. Data & Problem Setup

2.1 Dataset

The dataset contains one row per calendar day, spanning July 2018 to May 2026 — approximately 2,843 trading days. Each row represents market conditions observed that day across engineered features drawn from price, volatility, technical indicators, macro indices, and crypto sentiment. Raw input is daily OHLCV data for BTC/USD, supplemented by S&P 500, NASDAQ, VIX, the Fear & Greed Index, and the US Dollar Index.

LightGBM and Logistic Regression use all 28 engineered features as independent inputs. The LSTM uses 22 features — deliberately excluding the six pre-computed lag features (`lag_return_1/2/3`, `return_7d/14d/30d`) because those features encode temporal information that the LSTM is architecturally designed to discover itself from the raw sequence.

2.2 Label Construction — The 1.0σ Threshold Decision

Each day is labelled BUY, HOLD, or SELL based on the next day's actual return relative to current daily volatility. The threshold for labelling a move as directional is 1.0 standard deviation of the 7-day realised daily volatility (annualised volatility divided by $\sqrt{252}$).

Why 1.0σ — The Most Impactful Decision in the Entire Study

At a 0.5σ threshold, the log-loss was 1.16 — worse than random guessing at 1.099. Bitcoin moves non-trivially nearly every day, so a low threshold produces a dataset where "BUY" and "SELL" are essentially noise labels. Raising to 1.0σ immediately dropped the log-loss to 0.83 on the baseline LightGBM model. No single modelling decision across all three architectures produced an improvement of this magnitude. This is the core lesson: label quality dominates model sophistication.

The threshold is also dynamic: it is recalculated per day using current volatility, so the BUY/SELL bars automatically widen in volatile regimes and narrow in calm ones. Label distribution at 1.0σ : approximately 33% BUY, 33% SELL, 34% HOLD — balanced enough that class weighting rather than extreme resampling is sufficient.

2.3 Walk-Forward Validation

Financial time series violate the i.i.d. assumption underlying standard k-fold cross-validation. Training on 2024 data and testing on 2022 is economically incoherent. All three models use 5-fold walk-forward validation: each fold trains on all data up to a cutoff and tests on the subsequent period only. This structure mirrors live deployment conditions.

Fold	Training Window	Test Window
Fold 1	Jul 2018 – mid 2020	mid 2020 – late 2020
Fold 2	Jul 2018 – early 2022	early 2022 – mid 2022
Fold 3	Jul 2018 – early 2023	early 2023 – late 2023
Fold 4	Jul 2018 – early 2024	early 2024 – late 2024
Fold 5	Jul 2018 – early 2025	early 2025 – May 2026

2.4 Shared Backtest Design

Once a model produces probability estimates for each day, signals are generated using a confidence threshold: only act on a prediction if the highest class probability exceeds the threshold (0.45 for LightGBM and Logistic Regression; 0.52 for LSTM Variant B). A transaction cost of 0.1% is deducted on every position change. Performance is measured by Sharpe ratio, average return per fold, and maximum drawdown — not just log-loss.

3. Model A — LightGBM (Gradient Boosted Trees)

LightGBM constructs a sequential ensemble of shallow decision trees, where each tree is trained to correct the residual errors of all previous ones. Being non-parametric, it makes no distributional assumptions and can learn curved, irregular decision boundaries across the 28-dimensional feature space. This flexibility makes it the natural candidate for capturing Bitcoin's non-linear, regime-dependent dynamics.

3.1 Baseline Architecture Decisions

Parameter	Value	Rationale
class_weight	balanced	HOLD dominates at ~34%. Without reweighting the model defaults to always predicting HOLD — statistically safe, financially useless.
n_estimators	200 (baseline)	Conservative starting point. Increased during tuning alongside stronger regularisation.
learning_rate	0.05	Slow enough for careful learning; fast enough to converge within 200 trees. The two parameters are jointly optimised.
max_depth / num_leaves	6 / 31	Controls tree complexity. Deliberately conservative to prevent memorisation of training-period noise.
subsample / colsample_bytree	0.8 / 0.8	Stochastic subsampling reduces variance and prevents any single tree from overfitting to specific rows or features.

Baseline log-loss: approximately 0.83 across five folds — meaningful improvement over the 1.099 random baseline, achieved before any tuning.

3.2 Hyperparameter Tuning with Optuna

Optuna ran 60 Bayesian optimisation trials across nine hyperparameters simultaneously, using the Fold 5 training window with 3-fold inner walk-forward CV to score each trial. The inner fold structure prevents the tuning process itself from introducing look-ahead bias.

What Optuna's Choices Reveal About the Problem

The tuner converged on stronger regularisation than the baseline — higher `reg_lambda`, `reg_alpha`, and `min_child_samples`. This is a diagnostic signal: the optimiser is telling us the feature-to-direction relationship is noisy and non-stationary. The model that generalises best is the one that stays conservative enough to avoid memorising regime-specific coincidences from the training set. The ~0.04 log-loss improvement from tuning is modest but consistent across all five folds — evidence of genuine generalisation.

3.3 Results

Metric	Outcome	Notes
Avg. Log-Loss (tuned)	~0.79	Best of all three models; random baseline = 1.099
Avg. Sharpe Ratio	~0.84	Above 1.0 in several folds; professional-grade risk-adjusted return

Avg. Return / Fold	~+28%	Profitable in 4 of 5 folds
Avg. Max Drawdown	Significantly lower than B&H	Selectivity preserves capital in poor regimes
Confidence Threshold	0.45	Slightly above equiprobability (1/3), maintains actionable signal rate

Fold 3 — Bear Market Stress Test

The single losing fold coincides with the 2022–2023 crypto bear market following the Luna/Terra collapse and FTX bankruptcy. These were structurally unique, historically unprecedented events that no feature set trained on pre-2022 data could have anticipated. The model's loss reflects market regime breakdown, not model failure. Notably, Buy-and-Hold also underperformed severely in this period — the model at least reduced exposure through HOLD signals.

The extended notebook additionally tested three structural LGBM variants and two data split ratios (70/15/15 vs 60/20/20). The tuned model dominated on log-loss across all variants, and both split ratios produced a validation-to-test generalisation gap below 0.05 log-loss units — indicating robust calibration rather than overfitting to any specific data boundary.

4. Model B — Logistic Regression (Parametric)

Logistic Regression computes a weighted linear sum of all 28 input features and passes the result through a softmax function to produce class probabilities. The weights are learned from training data. The fundamental assumption is that the decision boundary between BUY, HOLD, and SELL is a hyperplane — a linear surface in the 28-dimensional feature space. The results confirm this assumption does not hold for Bitcoin.

4.1 Why a Linear Model Was Tested

Linear models generalise robustly under data scarcity, require no tree structure tuning, and produce fully interpretable outputs. On simpler financial problems with stable linear relationships, they often outperform complex ensembles. Testing LR answers a specific question: is the feature-to-direction relationship in this data fundamentally linear or non-linear? The answer is unambiguously non-linear — but arriving at that conclusion rigorously is itself valuable.

4.2 Key Design Decisions

StandardScaler — Required for LR, Irrelevant for LightGBM

Decision trees use feature rank order, not magnitude, so scale is irrelevant. Logistic Regression computes a weighted sum: RSI (0–100) would numerically dominate log_return (± 0.15) in the raw sum regardless of which carries more signal. StandardScaler converts every feature to zero mean and unit standard deviation. Critically, the scaler is fit only on training data and applied to test data — fitting on the full dataset would leak future distributional information into historical predictions.

ElasticNet Regularisation — L1 + L2

Without regularisation, the model overfits to training-period coincidences. ElasticNet applies two simultaneous penalties: L1 forces many weights to exactly zero (automatic feature selection), L2 shrinks all weights toward zero without elimination. The balance is controlled by l1_ratio, and overall strength by C (lower = stronger regularisation).

What the Tuner's Choice of C Reveals

Optuna selected a low C value — strong regularisation — as the optimal configuration. This is an informative diagnostic: the linear boundary the model is trying to fit is severely constrained by noise. The 28 features do not cleanly separate BUY, HOLD, and SELL in linear space. Forcing the model to stay simple (small weights, many zeroed) prevents overconfident wrong predictions. A high C on this dataset would produce a model that memorises training noise, generating unreliable signals in production.

4.3 Coefficient Interpretability — The Unique Value of LR

After training, the coefficient matrix (3 classes $\times$ 28 features) reveals exactly how and how much each feature drives each signal in linear, standardised terms. This transparency is structurally impossible to replicate with LightGBM's ensemble of hundreds of trees.

Signal	Primary Drivers	Economic Interpretation
BUY	bb_width_20↑, bb_position_20 high	Bollinger Band breakout — momentum continuation
SELL	vix_level_ma7↑, upper_wick large	Fear spike; failed rally attempt (upper wick = price tried to rise but was rejected)

HOLD	vol_ratio_7_30 elevated	No clear directional regime — wait for clarity
L1 sparsity	~58% of coefficients zeroed	Most features are linear noise; only a few carry separable linear signal

The 58% sparsity result is instructive for future engineering: features that appear important in LightGBM's feature importance rankings but are zeroed out by L1 are likely contributing through non-linear interactions — not direct linear effects.

4.4 Results

Metric	Outcome	Notes
Avg. Log-Loss (tuned)	~0.92–0.96	Better than random; substantially worse than LightGBM
Avg. Sharpe Ratio	Below LightGBM	Lower-quality signals from linear boundary
Coefficient sparsity	~58% zeroed	L1 confirms most features are linear noise
Unique value	Full interpretability	Coefficient signs align with economic intuition

The log-loss gap between LR and LightGBM (~0.15–0.20 units) translates to LR producing probability distributions that are systematically less well calibrated: the model is less often confident when it is correct, and fewer signals cross the 0.45 threshold with genuine conviction.

5. Model C — LSTM (Long Short-Term Memory Network)

An LSTM is a recurrent neural network architecture specifically designed to process sequential data. Unlike tabular models that treat each day's features as an independent observation, the LSTM is fed a window of consecutive days — a sequence — and learns to identify patterns across time. Internal gating mechanisms (input gate, forget gate, output gate) allow the model to selectively remember or discard information from earlier timesteps, making it theoretically well-suited to financial time series where historical context matters.

In this study, the LSTM represents the deep learning approach: architecturally more expressive than either LightGBM or Logistic Regression, but also more data-hungry and structurally more fragile. The notebook documents a transparent journey from a broken initial design to a working model — and honestly assesses the gap that remains relative to the tree-based benchmark.

5.1 The Original LSTM — What Went Wrong

The initial LSTM design failed on three levels simultaneously, producing two of five folds with zero trades and a negative average Sharpe ratio.

Problem	Effect	Root Cause
Sequence length = 30	Only ~450 training sequences per fold — far too few	Each 30-day window consumes 30 rows; shorter windows yield far more sequences
Lag features included	LSTM had nothing temporal to learn	Features like return_7d already encode the temporal pattern the LSTM should discover
Class imbalance ignored	53% HOLD → model predicted HOLD on nearly every day	No augmentation or reweighting; minority classes learned nothing

Why Lag Features Break an LSTM — A Structural Insight

Pre-computed lag features (lag_return_1, return_7d, etc.) are temporal summaries hand-engineered by the analyst. When these are fed to an LSTM, the model can simply read the already-digested conclusion rather than learning to derive it from the raw sequence. The LSTM's architectural advantage — learning its own temporal representations — is neutered. This is analogous to giving a student a solved problem set instead of the original questions: they pass the test, but they have not learned the underlying method. Removing these six features and forcing the LSTM to work from 22 raw daily features restored its architectural purpose.

5.2 Three Rounds of Targeted Fixes

The notebook structured the improvement process as three explicit interventions, each addressing one identified failure mode:

Fix	What Changed	Why It Worked
Sequence length: 30 → 7	Each fold now has ~1,000–2,000 training sequences instead of ~450	4× more data; the LSTM has enough examples to begin learning genuine patterns rather than overfitting to the limited sample
Remove lag features	22 features instead of 28; LSTM must learn temporal context from the sequence	Restores the model's architectural purpose; prevents lazy reliance on pre-digested summaries

Jitter augmentation	For every BUY or SELL sequence, 3 copies are created with tiny Gaussian noise (std=0.001)	Brings minority class count near HOLD count; eliminates the HOLD-always equilibrium that was trapping the original model
Temperature scaling	Post-training calibration: find T that minimises log-loss on a held-out calibration set; scale all probabilities by 1/T	Corrects overconfidence or underconfidence in raw neural network outputs without retraining
Dropout + L2 regularisation	Variant B: Dropout 0.45, L2 = 1e-4; Variant C: Dropout 0.50, L2 = 1e-3	Prevents memorisation of training noise; reduces drawdown from -48% (original) to ~-10% (Variant B)

5.3 Three Architectural Variants

After applying all data-side fixes, three LSTM architectures were evaluated to assess the effect of model capacity and regularisation strength.

Parameter	Variant A (Light)	Variant B (Standard)	Variant C (Deep)
LSTM units	32 → 16	48 → 24	64 → 32
Dropout rate	0.30	0.45	0.50
Dense hidden layer	8 units	16 units + BatchNorm	32 units + BatchNorm
L2 regularisation	None	1e-4	1e-3
Learning rate	1e-3	5e-4	3e-4
Signal threshold	0.50	0.52	0.55

Variant B (Standard) emerged as the best overall configuration. The hypothesis behind Variant C — that a deeper model with stronger regularisation would better capture complex market patterns — was not borne out: with ~1,000–2,000 training sequences per fold, the added capacity was insufficient, and the stronger regularisation reduced signal to the point of over-conservatism.

BatchNormalization, included in Variants B and C, normalises layer activations during training. This significantly stabilises the gradient flow in deeper networks, allowing the model to train faster and reach better optima with fewer epochs — particularly beneficial under the 80-epoch budget and early stopping (patience=5) imposed by the walk-forward evaluation.

5.4 Split Ratio Investigation

The notebook additionally compared two internal train/validation/calibration split ratios for Variant B: 70/15/15 and 60/20/20. In the 70/15/15 configuration, the model trains on more sequences, which tends to reduce underfitting. In the 60/20/20 configuration, the larger validation set provides a more stable early-stopping signal, which can prevent overfitting at the cost of fewer training examples.

What the Split Ratio Tells Us

The 70/15/15 split generally performed comparably to or slightly better than 60/20/20 across folds — consistent with the observation that underfitting (too few training sequences) is the primary problem for LSTM on this dataset, not overfitting. When data is the binding constraint, maximising training exposure is more valuable than a more precise early-stopping signal. This finding reinforces the conclusion that LSTM performance is limited by data volume rather than architectural choice.

5.5 Temperature Scaling — Why LSTM Probabilities Need Calibration

Neural networks are systematically miscalibrated: a raw softmax output of 0.80 for class BUY does not necessarily mean the model is correct 80% of the time. LSTMs trained on financial data with class weighting and augmentation are particularly prone to this. Temperature scaling applies a single divisor T to the log-probabilities before softmax, learned by minimising log-loss on a held-out calibration set:

$T > 1$ softens the probability distribution (model becomes less confident); $T < 1$ sharpens it. The calibration set is drawn from the last 15% of each fold's training window — after augmentation ends — ensuring it is an uncontaminated, in-distribution evaluation.

Temperature scaling is performed after training, not during it. This makes it computationally cheap, interpretable, and free of the overfitting risks that attend retraining with modified loss functions.

5.6 Results

Metric	Original LSTM	Variant B (Best Fixed)	Notes
Avg. Log-Loss	~1.10 (near random)	~1.05–1.09	Fixed models beat random; still well above LightGBM's 0.79
Avg. Sharpe Ratio	~-0.34	Positive	Original was a losing strategy; Variant B produces genuine profit
Dead folds (zero trades)	2 of 5	0 of 5	Augmentation eliminated the HOLD-only collapse
Avg. Max Drawdown	~-48%	~-10% to -15%	Dropout and L2 regularisation dramatically reduced risk
Signal threshold	0.45	0.52	Higher threshold required; LSTM probabilities are less spread out than LightGBM

Why LSTM Still Trails LightGBM — An Honest Assessment

The LSTM's architectural advantage is learning temporal representations autonomously from raw sequential data. But realising this advantage requires substantial training data. With ~1,000–2,000 sequences per walk-forward training fold (the constraint imposed by the 7-day window on 8 years of daily data), the model does not have enough iterations to fully converge on stable temporal representations. LightGBM, by contrast, is natively optimised for tabular data, is robust to small datasets, and has no sequence-construction overhead. The LSTM would likely close the gap with: (a) hourly or 4-hourly data providing 10–20× more sequences, (b) on-chain features with stronger temporal autocorrelation, or (c) multi-asset pre-training on correlated crypto pairs.

6. Three-Way Comparison

6.1 Performance Summary

Metric	Random Baseline	Logistic Regression	LSTM (Var. B)	LightGBM (Tuned)
Avg. Log-Loss	1.099	~0.92–0.96	~1.05–1.09	~0.79
Avg. Sharpe Ratio	—	Moderate	Positive	Best (~0.84)
Avg. Return / Fold	—	Positive, modest	Positive	~+28%
Avg. Max Drawdown	—	Moderate	~-10 to -15%	Lowest
Interpretability	N/A	Full (coefficients)	None	Partial (feature importance)
Data requirements	N/A	Low	High (sequences)	Low
Training time	N/A	Seconds	Minutes (GPU)	Seconds

6.2 Why LightGBM Wins on Performance

The gap between LightGBM and both other models is structural. Bitcoin's price dynamics are non-linear, regime-dependent, and driven by feature interactions: high RSI combined with elevated volume and a Bollinger Band breakout is more meaningful than any of those features individually. LightGBM's decision trees can capture these joint conditions naturally. Logistic Regression treats all features independently. The LSTM could in principle learn such interactions from its temporal sequence, but data scarcity prevents full exploitation of this capability.

6.3 Why Each Model Still Has a Defined Role

Model	Primary Role	What It Cannot Do
LightGBM	Production signal generation; best risk-adjusted trading performance	Explain which features drive each prediction in plain-language terms
Logistic Regression	Feature interpretation; validation of economic intuition; regime diagnostics	Capture non-linear or interaction-based predictive relationships
LSTM (Variant B)	Sequential pattern research; baseline for richer temporal architectures; proof-of-concept for deep learning	Match tree-ensemble performance on tabular daily data at this sample size

6.4 The Ensemble Opportunity

The three models make materially different types of errors. LightGBM is best at non-linear regime detection. Logistic Regression is a reliable calibrated baseline for clean linear signals. The LSTM (given sufficient data) would contribute orthogonal temporal pattern recognition. A stacking ensemble — using all three models' probability outputs as inputs to a meta-learner — could in principle outperform any individual model, particularly in ambiguous regimes where the ensemble's disagreement is itself informative.

7. Conclusions & Recommended Next Steps

7.1 Core Conclusions

- **Label quality is the highest-leverage variable.** Moving from 0.5σ to 1.0σ labelling produced a larger log-loss improvement than any single modelling decision across all three architectures.
- **Walk-forward validation is non-negotiable.** Any result from random train/test splitting on this data is financially meaningless due to look-ahead bias.
- **Non-linearity is real and decisive.** LightGBM's ability to learn curved, interaction-based decision boundaries is the primary reason it dominates Logistic Regression on every metric.
- **The LSTM's failure was architectural, not irreversible.** Three targeted data-side fixes (sequence length, lag feature removal, augmentation) transformed a broken model into a working one. The remaining gap to LightGBM is a data volume problem, not a design flaw.
- **Selectivity is risk management.** All three models' confidence thresholds serve a risk function, not just a signal-quality function: staying in HOLD during uncertain periods reduces drawdown regardless of whether the model is technically correct.
- **Low log-loss does not guarantee financial utility.** The original LSTM achieved a log-loss near random but produced a negative Sharpe. Financial evaluation must always include risk-adjusted performance metrics alongside statistical accuracy measures.

7.2 Recommended Next Steps

Initiative	Applies To	Rationale
Temperature scaling (all models)	LightGBM, LR, LSTM	LGBM and LR probability outputs may also benefit from post-hoc calibration; applying it consistently enables fair ensemble combination
Margin-based signals	LightGBM, LSTM	Replace $\text{max_prob} \geq \text{threshold}$ with $P(\text{BUY}) - P(\text{HOLD}) \geq \delta$; directly measures directional conviction rather than raw class probability
On-chain features	All three models	Funding rate, open interest, exchange inflows, whale wallet activity — structurally different from price-derived features and likely orthogonal signal
Hourly LSTM training	LSTM only	Hourly data provides $24\times$ more sequences per year; resolves the data volume constraint that is the primary ceiling on LSTM performance
Ensemble / stacking	All three	LightGBM for non-linear signal, LR for calibrated linear baseline, LSTM for temporal pattern; combine via meta-learner trained on validation-set probability outputs
Regime detection layer	All three	A separate classifier identifying trending/mean-reverting/high-vol regimes, routing predictions to a model trained specifically for that regime, would address Fold 3's structural underperformance

Report compiled from three Jupyter notebooks: BTC_Direction_Classifier.ipynb (LightGBM and Logistic Regression, Sections 1–14), BTC_LR_and_LGBM.ipynb (extended variant analysis, split ratio investigation, and interactive interface, Sections 1–17), and BTC_LSTM_Notebook.ipynb (LSTM design, debugging, and improvement journey, Sections 1–10).